claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_736be611-fb5\agent-a1a2dadd681346e65.jsonl`
- SHA-256: `334ac8d5600933731d89a3ca01d0bb26d91798ad6c574f09de9a0b5d2c77ef9d`
- Source modified: `2026-06-16T15:14:22+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_736be611-fb5`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`

Transcript

1. user (2026-06-16T15:12:42.308Z)

You are a racket-sports shuttle-trajectory signal expert. WHY does the SAME windowing over-segment the GX court ~2x but fit mahadevpura-2, and leave mahadevpura-1 as one continuous blob? Reason about court/camera/rally-density differences, the "shuttle moving != rally" conflation, and especially the GROUND-TRUTH finding that local's rally-START is ~at par with Gemini (|?start| 1.74 vs 1.46s) while the rally-END is the gap (|?end| 3.12 vs 1.19s). What trajectory-derived signal best fixes rally-END detection (e.g. sustained inactivity / See et al. >=80% no-motion-frames rule)? Cite numbers.

REAL-FOOTAGE VALIDATION DATA (6 Kushagra/Yash June-12 clips, 1080p proxies, fully-LOCAL no-AI WASB
windowing vs Gemini. Gemini = strong REFERENCE, NOT ground truth ? these 6 have no golden labels EXCEPT mahadevpura-2 which the owner hand-labeled, see GROUND-TRUTH below).

Rally COUNT (Gemini | local baseline | local W1 cap=12 | local W1+W2 mc=1):
  GX010128: 39 | 15 | 78 | 75
  GX010129: 27 | 2 | 50 | 47
  GX020125: 11 | 6 | 18 | 15
  mahadevpura-0: 40 | 19 | 62 | 43
  mahadevpura-1: 9 | 1 | 2 | 2
  mahadevpura-2: 39 | 5 | 40 | 40
  AGGREGATE: Gemini 165 | baseline 48 (mean window 62.9s, 27 merge-groups, mIoU 0.23) |
             W1 250 (mean 10.8s, 10 merge-groups, mIoU 0.39) |
             W1+W2 222 (mean 11.5s, 10 merge-groups, mIoU 0.39, 6 gemini-only "misses")
  Per-video mean window (W1): GX010128 7.8s, GX010129 9.3s, GX020125 14.1s, mahadevpura-0 10.4s,
             mahadevpura-1 86.9s (the continuously-active blob W1 cannot cap ? no inactivity gap),
             mahadevpura-2 13.8s. Gemini means ~5.7-7.2s.

GROUND-TRUTH (mahadevpura-2, 40 owner-labeled rallies, mean 5.7s, verified frame-aligned):
  F1@0.5 / F1@0.25 / mean-IoU: Gemini 0.557/0.835/0.55 ; local W1+W2 0.150/0.500/0.31 ;
  local W1+W2+hard-cap 0.296/0.722/0.50 (best local). baseline 0.000/0.000/0.03.
  BOUNDARY ERROR (matched rallies, seconds): Gemini |?start| med 1.46 |?end| med 1.19 ;
  local-hardcap |?start| med 1.74 (~= Gemini!) |?end| med 3.12 (>> Gemini).
  => local's rally-START detection is ~at par with Gemini; the gap is the rally-END.
  IoU<->time: for a 5.7s rally, tIoU 0.5 tolerates only ~+-1.9s shift (the ~1-1.5s service window
  nearly exhausts it -> tIoU 0.5 over-penalizes; owner wants a +-2s tolerance metric).

PRIOR GOLDEN DELTAS (n=6, then n=7 incl. mahadevpura-2): baseline F1 0.300 -> W1 0.439 (+0.139, 6/0)
  -> W1+W2 0.457 (+0.019, 6/0); hard-cap golden-neutral on n=6 (+0.006 n.s.) but became BEST on n=7
  (continuously-active clips). All flags default-OFF, promoted only on measured LOVO lift +

sign-test.

WINDOWING KNOBS: a frame is "active" iff shuttle visible AND per-second-normalized velocity > velocity_thresh(0.02); active frames within merge_gap(2.0s) coalesce; min_window(2.0s). W1 = max_window_duration cap(12s) split at the largest internal inactivity gap. W1.5 hard-cap = midpoint-chop when no gap. W2 = net-crossings gate (drop windows with <1 net crossing).

For raw per-video detail you MAY Read C:/Users/avidu/Projects/badminton-highlight-indexer/output/local_vs_gemini_6.json . Treat the DATA block as authoritative for the numbers. Constraints: local pipeline must stay MIT/Apache/BSD-only, single local GPU, ~6-7 golden videos now (~16 by the weekend), Gemini usable at inference only (never as a training-data source).

2. assistant (2026-06-16T15:12:45.621Z)

I'll analyze the over-segmentation behavior and the rally-END detection gap. Let me first read the raw per-video data to ground my reasoning in the actual numbers.

3. user (2026-06-16T15:12:46.384Z)


```
[
  {
    "name": "GX010128",
    "vid": "db84f3e65318",
    "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010128__db84f3e65318_wasb.csv",
    "n_gem": 39,
    "play_gem": 228.0,
    "mean_gem": 5.846153846153846,
    "variants": {
      "baseline": {
        "n": 15,
        "play": 711.4614614614616,
        "mean_dur": 47.430764097430774,
        "max_dur": 133.2499165832499,
        "count_ratio": 0.38461538461538464,
        "matched": 4,
        "local_only": 1,
        "gemini_only": 0,
        "splits": 0,
        "merges": 10,
        "mean_iou": 0.3564105364413525,
        "gem_rallies_in_merges": 35
      },
      "W1": {
        "n": 78,
        "play": 605.1051051051056,
        "mean_dur": 7.757757757757765,
        "max_dur": 16.616616616616625,
        "count_ratio": 2.0,
        "matched": 35,
        "local_only": 34,
        "gemini_only": 0,
        "splits": 4,
        "merges": 0,
        "mean_iou": 0.5294639963484731,
        "gem_rallies_in_merges": 0
      },
      "W1+W2": {
        "n": 75,
        "play": 594.494494494495,
        "mean_dur": 7.926593259926601,
        "max_dur": 16.616616616616625,
        "count_ratio": 1.9230769230769231,
        "matched": 35,
```

```
              "local_only": 31,
              "gemini_only": 0,
              "splits": 4,
              "merges": 0,
              "mean_iou": 0.5294639963484731,
              "gem_rallies_in_merges": 0
            }
          },
          "status": "ok"
        },
        {
          "name": "GX010129",
          "vid": "99a3e73afde3",
          "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\GX010129__99a3e73afde3_wasb.csv",
          "n_gem": 27,
          "play_gem": 195.6,
          "mean_gem": 7.2444444444444445,
          "variants": {
            "baseline": {
              "n": 2,
              "play": 534.4177510844178,
              "mean_dur": 267.2088755422089,
              "max_dur": 522.6893560226894,
              "count_ratio": 0.07407407407407407,
              "matched": 0,
              "local_only": 1,
              "gemini_only": 0,
              "splits": 0,
              "merges": 1,
              "mean_iou": 0.0,
              "gem_rallies_in_merges": 27
            },
            "W1": {
              "n": 50,
              "play": 463.7137137137137,
              "mean_dur": 9.274274274274273,
              "max_dur": 27.84451117784451,
              "count_ratio": 1.8518518518518519,
              "matched": 23,
              "local_only": 18,
              "gemini_only": 0,
              "splits": 4,
              "merges": 0,
              "mean_iou": 0.4819655005521238,
              "gem_rallies_in_merges": 0
            },
            "W1+W2": {
              "n": 47,
              "play": 447.0303636970303,
              "mean_dur": 9.511284333979368,
              "max_dur": 27.84451117784451,
              "count_ratio": 1.7407407407407407,
              "matched": 22,
              "local_only": 16,
              "gemini_only": 1,
              "splits": 4,
              "merges": 0,
              "mean_iou": 0.48514996335815863,
              "gem_rallies_in_merges": 0
            }
          },
          "status": "ok"
        },
        {
```

```
        "name": "GX020125",
        "vid": "407fa7e7e9f1",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\GX020125__407fa7e7e9f1_wasb.csv",
        "n_gem": 11,
        "play_gem": 71.0,
        "mean_gem": 6.454545454545454,
        "variants": {
          "baseline": {
            "n": 6,
            "play": 280.6806806806807,
            "mean_dur": 46.78011344678012,
            "max_dur": 82.66599933266599,
            "count_ratio": 0.5454545454545454,
            "matched": 1,
            "local_only": 2,
            "gemini_only": 0,
            "splits": 0,
            "merges": 3,
            "mean_iou": 0.40270862800565765,
            "gem_rallies_in_merges": 10
          },
          "W1": {
            "n": 18,
            "play": 254.03737070403744,
            "mean_dur": 14.113187261335414,
            "max_dur": 40.05672339005673,
            "count_ratio": 1.6363636363636365,
            "matched": 9,
            "local_only": 8,
            "gemini_only": 0,
            "splits": 0,
            "merges": 1,
            "mean_iou": 0.4329964346464835,
            "gem_rallies_in_merges": 2
          },
          "W1+W2": {
            "n": 15,
            "play": 246.5131798465132,
            "mean_dur": 16.434211989767547,
            "max_dur": 40.05672339005673,
            "count_ratio": 1.3636363636363635,
            "matched": 9,
            "local_only": 5,
            "gemini_only": 0,
            "splits": 0,
            "merges": 1,
            "mean_iou": 0.4329964346464835,
            "gem_rallies_in_merges": 2
          }
        },
        "status": "ok"
      },
      {
        "name": "mahadevpura-0",
        "vid": "6460c2fc7647",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\w2-safeguard\\output\\wasb\\mahadevpura-0__6460c2fc7647_wasb.csv",
        "n_gem": 40,
        "play_gem": 262.0,
        "mean_gem": 6.55,
        "variants": {
          "baseline": {
            "n": 19,
            "play": 711.5615615615616,
```

```
            "mean_dur": 37.45060850324008,
            "max_dur": 134.13413413413414,
            "count_ratio": 0.475,
            "matched": 6,
            "local_only": 4,
            "gemini_only": 0,
            "splits": 0,
            "merges": 9,
            "mean_iou": 0.3446682872977475,
            "gem_rallies_in_merges": 34
          },
          "W1": {
            "n": 62,
            "play": 646.1127794461132,
            "mean_dur": 10.421173862034085,
            "max_dur": 47.66433099766434,
            "count_ratio": 1.55,
            "matched": 34,
            "local_only": 22,
            "gemini_only": 0,
            "splits": 2,
            "merges": 2,
            "mean_iou": 0.4928109196809172,
            "gem_rallies_in_merges": 4
          },
          "W1+W2": {
            "n": 43,
            "play": 549.5161828495164,
            "mean_dur": 12.779446112779452,
            "max_dur": 47.66433099766434,
            "count_ratio": 1.075,
            "matched": 30,
            "local_only": 9,
            "gemini_only": 5,
            "splits": 1,
            "merges": 2,
            "mean_iou": 0.4837616679798798,
            "gem_rallies_in_merges": 4
          }
        },
        "status": "ok"
      },
      {
        "name": "mahadevpura-1",
        "vid": "38bef75d78cb",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-1__38bef75d78cb_wasb.csv",
        "n_gem": 9,
        "play_gem": 51.2,
        "mean_gem": 5.688888888888889,
        "variants": {
          "baseline": {
            "n": 1,
            "play": 174.2742742742743,
            "mean_dur": 174.2742742742743,
            "max_dur": 174.2742742742743,
            "count_ratio": 0.1111111111111111,
            "matched": 0,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 1,
            "mean_iou": 0.0,
            "gem_rallies_in_merges": 9
          },
```

```
          "W1": {
            "n": 2,
            "play": 173.79045712379047,
            "mean_dur": 86.89522856189524,
            "max_dur": 101.88521855188523,
            "count_ratio": 0.2222222222222222,
            "matched": 0,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 2,
            "mean_iou": 0.0,
            "gem_rallies_in_merges": 9
          },
          "W1+W2": {
            "n": 2,
            "play": 173.79045712379047,
            "mean_dur": 86.89522856189524,
            "max_dur": 101.88521855188523,
            "count_ratio": 0.2222222222222222,
            "matched": 0,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 2,
            "mean_iou": 0.0,
            "gem_rallies_in_merges": 9
          }
        },
        "status": "ok"
      },
      {
        "name": "mahadevpura-2",
        "vid": "038e0e0915a2",
        "csv": "C:\\Users\\avidu\\Projects\\badminton-highlight-
indexer\\.claude\\worktrees\\local-noai\\output\\wasb\\mahadevpura-2__038e0e0915a2_wasb.csv",
        "n_gem": 39,
        "play_gem": 266.0,
        "mean_gem": 6.82051282051282,
        "variants": {
          "baseline": {
            "n": 5,
            "play": 605.9059059059059,
            "mean_dur": 121.18118118118119,
            "max_dur": 297.03036369703034,
            "count_ratio": 0.1282051282051282,
            "matched": 2,
            "local_only": 0,
            "gemini_only": 0,
            "splits": 0,
            "merges": 3,
            "mean_iou": 0.2945165777074605,
            "gem_rallies_in_merges": 37
          },
          "W1": {
            "n": 40,
            "play": 551.0844177510842,
            "mean_dur": 13.777110443777104,
            "max_dur": 43.3266599933267,
            "count_ratio": 1.0256410256410255,
            "matched": 12,
            "local_only": 4,
            "gemini_only": 0,
            "splits": 4,
            "merges": 5,
```

```
            "mean_iou": 0.4204774017180335,
            "gem_rallies_in_merges": 11
          },
          "W1+W2": {
            "n": 40,
            "play": 551.0844177510842,
            "mean_dur": 13.777110443777104,
            "max_dur": 43.3266599933267,
            "count_ratio": 1.0256410256410255,
            "matched": 12,
            "local_only": 4,
            "gemini_only": 0,
            "splits": 4,
            "merges": 5,
            "mean_iou": 0.4204774017180335,
            "gem_rallies_in_merges": 11
          }
        },
        "status": "ok"
      }
    ]
```


4. assistant (2026-06-16T15:12:49.939Z)

I have the authoritative data. Now let me reason through the mechanics rigorously. Let me look at the actual windowing/segmenter code to ground my claims about *why* the same knobs behave so differently.

5. user (2026-06-16T15:12:50.232Z)

```
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\fusion_hybrid.py
```

6. user (2026-06-16T15:12:54.106Z)

```
Found 5 files
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
```

7. user (2026-06-16T15:12:55.107Z)

```
Found 46 files
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\gemini.py
backend\reporting\harvest.py
backend\results.py
backend\utils\geometry.py
backend\pipeline\validators\format.py
backend\pipeline\validators\blur.py
backend\pipeline\detectors\audio_features.py
```

```
backend\pipeline\detectors\base.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\personalization\__init__.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\validators\base.py
backend\pipeline\detectors\__init__.py
backend\eval\golden_fixtures.py
backend\eval\per_user_eval.py
backend\eval\per_user_regression.py
backend\eval\regression.py
backend\eval\score_calibration.py
backend\eval\segmentation_metrics.py
backend\eval\windowing.py
backend\infrastructure\database.py
backend\infrastructure\execution_policy.py
backend\main.py
backend\eval\gemini_refine.py
backend\config\models.py
backend\eval\__init__.py
backend\eval\ablation.py
backend\eval\audio\__init__.py
backend\eval\classifier.py
backend\eval\eval_partial_labels.py
backend\eval\eval_stats.py
backend\eval\experiment.py
backend\eval\export_wasb_segments.py
backend\annotations\__init__.py
backend\annotations\base.py
```

8. assistant (2026-06-16T15:12:57.587Z)

The windowing logic lives in `backend/eval/windowing.py`. Let me read it to ground my analysis of W1/W1.5/W2 in the actual implementation.

9. user (2026-06-16T15:12:58.315Z)

```
1       """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3       **The bug this closes.** The offline eval substrate (``distill_local.build_candidates``
?
4       ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5       on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6       deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7       (``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02,
merge_gap=2.0,
8       min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set
is
9       measured on candidates **production never serves**, so a "win" need not transfer. This
is
10      exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11      (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).
12
13      **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14      SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
```

```
    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
    build candidates under either preset, and :func:`served_windowing_from_config` lifts the
    *actual* served windowing out of a live config (overrides included), so an A/B can
target the
    operating point a given deployment truly serves.

    The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
    windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
    ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.

    Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
    caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
    """
    from __future__ import annotations

    from dataclasses import dataclass
    from typing import Any, Dict, List, Mapping, Tuple

    from backend.config.models import IndexingConfig

    Interval = Tuple[float, float]

    __all__ = [
        "WindowingPreset",
        "SERVED",
        "PROPOSAL",
        "PRESETS",
        "served_windowing_from_config",
        "build_windows",
        "windows_to_intervals",
    ]


    @dataclass(frozen=True)
    class WindowingPreset:
        """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.

        ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
        "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
        ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
        knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
        """

        name: str
        velocity_thresh: float
        merge_gap: float
        min_window_duration: float

        def as_tuple(self) -> Tuple[float, float, float]:
            """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
            ``trajectory_to_action_windows`` / ``distill_local`` call sites already
speak."""
            return (self.velocity_thresh, self.merge_gap, self.min_window_duration)

        def to_dict(self) -> Dict[str, Any]:
```

```
68              return {
69                  "name": self.name,
70                  "velocity_thresh": self.velocity_thresh,
71                  "merge_gap": self.merge_gap,
72                  "min_window_duration": self.min_window_duration,
73              }
74
75
76      def _served_preset_from_defaults() -> WindowingPreset:
77          """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
78
79          ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
80            - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
81            - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
82            - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
83
84          Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
85          point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
86          so the two can never silently diverge again.
87          """
88          idx = IndexingConfig()  # construct with defaults ? the served operating point
89          return WindowingPreset(
90              name="served",
91              velocity_thresh=float(idx.wasb.velocity_threshold),  # == fusion/tracknet
default 0.02
92              merge_gap=float(idx.dead_time_merge_gap),
93              min_window_duration=float(idx.min_action_window_duration),
94          )
95
96
97      #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
98      #: only promotable if it holds up HERE ? the operating point real users get.
99      SERVED: WindowingPreset = _served_preset_from_defaults()
100
101     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
102     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
103     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
104     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
105     PROPOSAL: WindowingPreset = WindowingPreset(
106         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
107     )
108
109     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
110     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
111
112
113     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
114         """The SERVED windowing a *specific* live config serves (overrides honoured).
115
116         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
117         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
118         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
119         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
```

```
120         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
121         """
122         idx = _as_mapping(config, "indexing")
123         family_cfg = _as_mapping(idx, family)
124         return WindowingPreset(
125             name=f"served:{family}",
126             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
127             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
128             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
129         )
130
131
132     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
133         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
134         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
135         if obj is None:
136             return {}
137         if isinstance(obj, Mapping):
138             sub = obj.get(key, {})
139         elif hasattr(obj, "get"):
140             sub = obj.get(key, {})
141         else:
142             sub = getattr(obj, key, {})
143         return sub if isinstance(sub, Mapping) else {}
144
145
146     def build_windows(points: List[Any], fps: float, frame_width: float,
147                       preset: WindowingPreset) -> List[Dict[str, Any]]:
148         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
149
150         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
151         instead of three loose floats, so eval candidate-building and the served path
reference the
152         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
153         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
154
155         vt, mg, mwd = preset.as_tuple()
156         return TrackNetRunner.trajectory_to_action_windows(
157             points, fps=fps, frame_width=frame_width,
158             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
159
160
161     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
162         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
163         return [(w["start"], w["end"]) for w in windows]
164
```

10. assistant (2026-06-16T15:13:02.133Z)

Now let me see the actual window-building algorithm (the active-frame coalescing + the W1 max-window split logic) in `tracknet_runner.py`.

11. user (2026-06-16T15:13:03.415Z)

```
backend\pipeline\detectors\tracknet_runner.py:142:        # Normalize separators so
basename/splitext work when tests (or collector)
```

```
backend\pipeline\detectors\tracknet_runner.py:146:        stem, ext = os.path.splitext(base)
backend\pipeline\detectors\tracknet_runner.py:256:    def trajectory_to_action_windows(
backend\pipeline\detectors\tracknet_runner.py:261:        velocity_thresh: float = 0.02,
backend\pipeline\detectors\tracknet_runner.py:262:        merge_gap: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py:265:        max_window_duration: float = 0.0,
backend\pipeline\detectors\tracknet_runner.py:266:        min_split_gap: float = 0.5,
backend\pipeline\detectors\tracknet_runner.py:272:        ``velocity_thresh`` ? i.e. the shuttle
is in flight. Active frames within
backend\pipeline\detectors\tracknet_runner.py:273:        ``merge_gap`` seconds of each other
are grouped into one window; short
backend\pipeline\detectors\tracknet_runner.py:277:        ``max_window_duration`` (0 = OFF, the
default ? behaviour unchanged): a guard against
backend\pipeline\detectors\tracknet_runner.py:280:        window longer than this is recursively
SPLIT at its largest internal inactivity gap (the
backend\pipeline\detectors\tracknet_runner.py:282:        that gap is at least ``min_split_gap``
seconds. This is the bounded-windowing fix from
backend\pipeline\detectors\tracknet_runner.py:303:                    if velocity >
velocity_thresh:
backend\pipeline\detectors\tracknet_runner.py:310:        max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\tracknet_runner.py:327:            if af[0] - group[-1][0] <=
max_gap_frames:
backend\pipeline\detectors\tracknet_runner.py:334:        if max_window_duration > 0:
backend\pipeline\detectors\tracknet_runner.py:335:            groups =
TrackNetRunner._split_overlong_groups(
backend\pipeline\detectors\tracknet_runner.py:336:                groups,
int(max_window_duration * fps), int(min_split_gap * fps))
backend\pipeline\detectors\tracknet_runner.py:342:    def _split_overlong_groups(groups:
List[List[Tuple[int, float]]], max_win_frames: int,
backend\pipeline\detectors\tracknet_runner.py:343:
min_split_frames: int) -> List[List[Tuple[int, float]]]:
backend\pipeline\detectors\tracknet_runner.py:344:        """Recursively split any active-frame
group longer than ``max_win_frames`` at its largest
backend\pipeline\detectors\tracknet_runner.py:345:        internal gap (? ``min_split_frames``)
? an inactivity-aware cut at the most likely rally
backend\pipeline\detectors\tracknet_runner.py:347:        invent or drop coverage; a group with
no gap ? the floor is left intact (a genuinely long
backend\pipeline\detectors\tracknet_runner.py:358:            best_i, best_gap = -1, -1
backend\pipeline\detectors\tracknet_runner.py:360:                gap = g[i][0] - g[i - 1][0]
backend\pipeline\detectors\tracknet_runner.py:361:                if gap > best_gap:
backend\pipeline\detectors\tracknet_runner.py:362:                    best_gap, best_i = gap, i
backend\pipeline\detectors\tracknet_runner.py:363:            if best_i < 0 or best_gap <
min_split_frames:
backend\pipeline\detectors\tracknet_runner.py:364:                out.append(g)  # too long but
no real dead-time gap ? keep as one rally
```

12. assistant (2026-06-16T15:13:05.991Z)

Let me read the full window-building and split functions to nail the exact mechanics.

13. user (2026-06-16T15:13:07.013Z)

```
282             prev: Optional[TrajectoryPoint] = None
283             for p in points:
284                 if not p.visible:
285                     prev = None  # break the trajectory; absent shuttle = not in flight
286                     continue
287                 if prev is not None:
288                     dframes = p.frame - prev.frame
289                     if dframes > 0:
290                         dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
291                         velocity = (dist / frame_width) * (fps / dframes)
292                         if velocity > velocity_thresh:
293                             active.append((p.frame, velocity))
294                 prev = p
295
```

```
296             if not active:
297                 return []
298 
299             max_gap_frames = int(merge_gap * fps)
300 
301             def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
302                 vels = [v for _, v in group]
303                 return {
304                     "start": group[0][0] / fps + chunk_start,
305                     "end": group[-1][0] / fps + chunk_start,
306                     "active_frame_count": len(group),
307                     "peak_velocity": max(vels),
308                     "mean_velocity": sum(vels) / len(vels),
309                     "track_id_count": 1,  # single shuttle; kept for window-shape parity
310                     "chunk_index": chunk_index,
311                 }
312 
313             windows = []
314             group = [active[0]]
315             for af in active[1:]:
316                 if af[0] - group[-1][0] <= max_gap_frames:
317                     group.append(af)
318                 else:
319                     windows.append(_finalize(group))
320                     group = [af]
321             windows.append(_finalize(group))
322 
323             return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
324 
```

14. assistant (2026-06-16T15:13:12.033Z)

Now I have the exact mechanics. The key insight from the code: a window's `end` is set to `group[-1][0] / fps` ? the timestamp of the **last active (in-flight) frame**. There is no trailing dead-time logic to extend the end to the true rally-stop, and critically, the `prev = None` reset on `not p.visible` means visibility dropouts break the velocity computation. Let me confirm the segmentation_metrics boundary-error computation and the See et al. reference, then check the fusion_hybrid for any existing inactivity rule.

15. user (2026-06-16T15:13:13.096Z)

```
Found 9 files
backend\eval\rally_seg_eval.py
backend\eval\golden_fixtures.py
backend\eval\harness.py
backend\eval\metrics.py
backend\eval\rally_seq_proto.py
backend\eval\served_gate_a.py
backend\eval\__init__.py
backend\eval\eval_partial_labels.py
backend\eval\export_wasb_segments.py
```

16. user (2026-06-16T15:13:14.656Z)

```
backend\pipeline\segmenters\fusion_hybrid.py:97:    # (owner/hardware) before any served
promotion. Flag-gated (default person OFF). See
```

17. assistant (2026-06-16T15:13:22.212Z)

No existing trailing-inactivity rule. I now have everything I need: the windowing mechanics, the boundary-error data, and confirmation that the end is set to the last in-flight frame. Let me verify the See et al. rule context isn't already referenced in docs, to cite it correctly rather than inventing provenance.

18. user (2026-06-16T15:13:23.758Z)

```
Found 10 files
docs\RALLY_QUALITY_RESEARCH.md
docs\QUALITY_ITERATIONS.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\ROORA_LABELING_GUIDELINES.md
docs\COMMERCIALIZATION.md
docs\EXPERIMENT_HARNESS.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\NEXT_STEPS.md
docs\ANALYZERS_AND_RECONCILERS.md
```

19. user (2026-06-16T15:13:28.718Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-37-| Heuristic velocity
windowing (local today) | 0.657 | **0.157** |
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-38-| Gen-0 owned head
(LogReg on 5 trajectory features, LOVO n=6) | 0.744 | 0.561 |
docs\RALLY_QUALITY_RESEARCH.md:39:| Two-stage WASB + Gemini refine | 0.83 | ? |
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-40-| **Gemini wrapper
(gemini-flash)** | **0.93** | **0.66** |
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-41-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-88-dead time the shuttle
is still visible and gently moving (being picked up, walked back, knock-ups,
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-89-slow serve prep) ?
those frames read as *active*. The 2 s merge then bridges the brief gaps, and
docs\RALLY_QUALITY_RESEARCH.md:90:because there is **no `max_window_duration` cap and no
inactivity/boundary split**, a single
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-91-window grows until the
shuttle finally goes invisible ? which, at 96?99 % visibility, is rarely.
docs\RALLY_QUALITY_RESEARCH.md-92-Result: rally + dead-time + rally + ? collapse into a few
mega-windows.
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-101-The local path **has
no rally-STATE signal** ? only "is the shuttle moving." Bridging the gap is
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-102-fundamentally about
giving L2 signals that separate *rally* from *shuttle-merely-moving*
docs\RALLY_QUALITY_RESEARCH.md:103:(net-crossings, two-sided play, sustained fast exchange, hit
sounds) and a segmenter that
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-104-produces *bounded,
well-cut* intervals ? exactly the multi-signal-fusion + learned-segmenter
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-105-direction the existing
docs lay out. This doc makes that concrete and measurable.
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-293-boundary heuristics
we're missing, in spirit license-compatible (re-confirm before vendoring code).
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-294-
docs\RALLY_QUALITY_RESEARCH.md:295:- **See et al. ? Service & End-of-Rally Detection in
Badminton** (2024 Springer chapter
```

docs\RALLY_QUALITY_RESEARCH.md-296-
[10.1007\978-3-031-69073-0_15](https://link.springer.com/chapter/10.1007/978-3-031-69073-0_15);
docs\RALLY_QUALITY_RESEARCH.md-297- 2025 SN Computer Science follow-on
[10.1007\s42979-025-03935-0](https://link.springer.com/article/10.1007/s42979-025-03935-0)).
docs\RALLY_QUALITY_RESEARCH.md:298: **(a) Rally END = a sustained-inactivity rule:** the rally
ends when **?80% of consecutive frames
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-299- show no shuttle
motion** (~**95% accuracy** over 187 rallies). **(b) Rally START (service) = a
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-300- learned classifier**
over court geometry + player poses (~26 features/frame × 12 frames ?
docs\RALLY_QUALITY_RESEARCH.md:301: CNN/BiLSTM, **88.6%** accuracy). **Why it fits:** the
inactivity rule is *precisely* the
docs\RALLY_QUALITY_RESEARCH.md-302- end-of-rally cut our windowing lacks ? a few lines,
directly breaks 65 s mega-windows. **? Caveats:**
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-303- the rule is a
termination, not a *within-rally split*; results rest on paywalled abstracts +
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-304- small test sets
(35?187 rallies); the serve classifier needs the (default-OFF) person cue + court
docs\RALLY_QUALITY_RESEARCH.md:305: polygons. **Effort/payoff:** the inactivity rule = **low-
effort high-confidence win**; serve
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-306- classifier = medium.
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-307-
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-319- scorer); full swing
fusion = medium.
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-320-
docs\RALLY_QUALITY_RESEARCH.md:321:- **What best separates "rally" from "shuttle merely
moving"** (synthesis): sustained-inactivity
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-322- termination (See),
hit cadence / direction-reversals (Hsu), net-crossings ?2 (our Gate-A,
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-323-
[`rally_gate.py`](../backend/pipeline/detectors/rally_gate.py)), two-sided court occupancy (our
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-396-### Tier 1 ? Low-
effort, high-confidence wins
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-397-- **W1 · Bounded
windowing (the direct over-merge fix).** Add a `max_window_duration` cap and a
docs\RALLY_QUALITY_RESEARCH.md:398: **sustained-inactivity split** to
`trajectory_to_action_windows` ? port See et al.'s rule (split /
docs\RALLY_QUALITY_RESEARCH.md:399: end a window when ?~80% of consecutive frames over a
trailing window show no shuttle motion).
docs\RALLY_QUALITY_RESEARCH.md-400- Config-gated, default preserves current behavior.
Success: segmental Edit + F1@k improve vs
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-401- `CONTROL` on the
golden set (?F1 CI excludes 0 **and** sign-test agrees) with **no served-windowing
docs\RALLY_QUALITY_RESEARCH.md:402: regression**; merge-rate drops materially. *(Win ? §5.2 See
et al., ~95% end-of-rally acc.)*
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-403-- **W2 · Rally-state
features into the learned scorer (no special-casing).** Wire the already-built,
docs\RALLY_QUALITY_RESEARCH.md-404- default-OFF cues as scorer feature columns: **net-
crossings** (Gate-A), **two-sided motion**, and a

```
--
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-415-
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-416-### Tier 2 ? Medium
effort
docs\RALLY_QUALITY_RESEARCH.md:417:- **M1 · Serve-start classifier ? rally-start boundary** (See
et al. pose/court, 88.6%). Needs the
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-418-  person cue + a
**court-mask source** (manual polygon vs auto-homography ? *owner-gated* per
docs\RALLY_QUALITY_RESEARCH.md-419-
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) §9.1). **Success:** start-boundary
--
C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\w2-
safeguard\docs\RALLY_QUALITY_RESEARCH.md-492-|---|---|---|---|---|---|
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-493-| **E1** |
merge/split-rate metric + F1@k surfaced + one-command `rally_seg_eval` entrypoint; **froze
baseline** (#185) | 0 | ? | over-merge shows up as a number on the 6 golden videos | ? baseline
F1 0.300, merge_rate 0.523 |
docs\RALLY_QUALITY_RESEARCH.md:494:| **W1** | Bounded windowing ? `max_window_duration` cap +
inactivity split (See et al.) (#187) | 1 | E1 | seg F1@k ? vs baseline (CI?0 **and** sign-test),
no served regression; merge-rate ? | ? **?F1 +0.139 (6/0), merge_rate 0.523?0.038** (default-
OFF) |
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-495-| **W2** | Rally-state
features (net-crossings ? #188 + **never-empty safeguard** ?; two-sided motion, **direction-
reversal/peak**, audio = follow-on) | 1 | E1, W1 | ?1 cue ?F1 CI clears 0 + sign-test, survives
served re-check ? promote; others retained default-OFF | ? **net-crossings ?F1 +0.019 (6/0)**,
gate now **do-no-harm** (never zeroes a video); multi-feature scorer next (GPU) |
C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\worktrees\w2-safeguard\docs\RALLY_QUALITY_RESEARCH.md-496-| **W3** | Keep scorer
parameter-light (standing decision; TemporalMaxer evidence) | 1 | ? | documented; no learned
temporal net until corpus ? now | ? |
```

20. assistant (2026-06-16T15:13:35.451Z)

I now have the full picture, grounded in code and the project's own research doc. The See et al. rule (?80% no-motion frames over a trailing window) is already documented as the intended W1 inactivity split. I have everything I need to answer rigorously. Let me synthesize.

Key mechanical finding from the code (`tracknet_runner.py:305`): a window's `end = group[-1][0] / fps` ? the timestamp of the **last in-flight frame**, with no trailing dead-time extension. Combined with `prev = None` on invisibility (line 285), the end fires whenever the shuttle goes invisible OR slow ? which is exactly the rally-END failure mode the data shows.

21. user (2026-06-16T15:14:12.989Z)

Structured output provided successfully

22. assistant (2026-06-16T15:14:22.592Z)

I've completed the analysis and returned the findings via StructuredOutput.

The core mechanism, grounded in `backend/pipeline/detectors/tracknet_runner.py`: a window's `end` is pinned to `group[-1][0]/fps` (line 305) ? the timestamp of the last frame with shuttle velocity > 0.02 ? with `prev` reset to `None` on any invisible frame (line 285). The W1 cap only ever splits at the *largest internal inactivity gap* and leaves a gap-less group intact (`_split_overlong_groups`, lines 358-364).

That single fact explains all three behaviors:
- **GX010128 ~2x over-seg**: short dense rallies (5.85s) with brief intra-rally shuttle stalls give the 12s cap a sub-rally gap to chop on ? 78 vs 39.

- **mahadevpura-2 fits**: inter-rally dead time produces clean gaps at true cadence ? 40 vs 39, best local F1@0.25 0.722.
- **mahadevpura-1 blob**: near-continuous visible motion, no gap ? `min_split_gap`, so the cap is a no-op ? n=2, mean 86.9s.

The ground-truth split confirms START is already at Gemini parity (|?start| 1.74 vs 1.46s) and the entire deficit is END (|?end| 3.12 vs 1.19s) ? because "shuttle moving ? rally" and the end fires on the first sub-threshold/invisible frame. The best fix is See et al.'s ?80%-no-motion trailing-window termination (~95% end-of-rally accuracy over 187 rallies; Springer 10.1007/978-3-031-69073-0_15), which is already the documented W1 design and MIT/Apache-clean to reimplement.